AI-Powered Fire Prediction Spread and Containment System

Coding Standards Document

Panic! At The Kernel
Client: EPI-USE Africa

August 2026



Full Name	Student Number
Megan Lai	u23608821
Inge Keyser	u23544563
Janri du Toit	u23632730
Ryan Lynn	u23541912
Marco de Wit	u22811975

Contents

1 Introduction **4**

2 Technology Stack **4**
2.1 Frontend . . . 4
2.2 Backend . . . 4
2.3 AI/ML . . . 4
2.4 Database . . . 4
2.5 Object Storage . . . 4
2.6 CI/CD . . . 4

3 Naming Conventions **4**
3.1 Frontend (React/TypeScript) . . . 5
3.2 Backend & AI/ML (Python) . . . 5

4 Code Layout & Structure **5**
4.1 Indentation . . . 5
4.2 Line Length . . . 5
4.3 Quotation Marks . . . 5
4.4 Braces . . . 5
4.5 Comments . . . 5
4.6 Imports (Python) . . . 5

5 Frontend Standards **6**
5.1 TypeScript . . . 6
5.2 DaisyUI (Styling) . . . 6
5.3 Playwright (Testing) . . . 6

6 Backend Standards **6**
6.1 Python . . . 6
6.2 Pytest (Testing) . . . 7

7 AI/ML Standards **8**

8 API Design Guidelines **8**

9 Error Handling & Logging **8**

10 Configuration & Secrets **8**

11 Configuration Management **9**

12 Code Reviews & Pull Requests **9**
12.1 Review Requirements . . . 9
12.2 Review Guidelines . . . 9

13 CI/CD Pipeline Standards **9**
13.1 GitHub Actions . . . 9
13.2 Deployment . . . 10

14 File Structure & Organisation **10**
14.1 Repository Structure . . . 10
14.2 File Naming Conventions . . . 11

15 Git Repository & Strategy **11**
15.1 Maintained Branches 11
15.2 Branch Purposes 11
15.3 Branch Names 12
15.4 Merging 12

16 Conclusion **12**

17 Change Log **13**

1 Introduction

This document describes the coding standards and conventions for Fire Away, an AI-Powered Fire Spread Prediction and Containment System. These standards will be used to ensure uniformity, clarity, flexibility, reliability, and efficiency in our code.

2 Technology Stack

2.1 Frontend

- React.js
- TypeScript
- DaisyUI (Styling)
- Playwright (Testing)

2.2 Backend

- Python
- Pylint (linting)
- Pytest (testing)

2.3 AI/ML

- Python
- PyTorch

2.4 Database

- PostgreSQL

2.5 Object Storage

- MinIO

2.6 CI/CD

- GitHub Actions

3 Naming Conventions

- Ensure names are descriptive and avoid abbreviations unless absolutely necessary.
- Keep conventions consistent. Do not mix camalCase and snake_case withing the same language.
- Avoid single-letter names with the exception of loop counters

3.1 Frontend (React/TypeScript)

- **Variables/Functions:** camelCase
- **React Components:** PascalCase
- **Custom Hooks:** camelCase
- **Types/Interface:** PascalCase
- **Constants:** UPPER_SNAKE_CASE

3.2 Backend & AI/ML (Python)

- **Variables/Functions:** snake_case
- **Classes:** PascalCase
- **Constants:** UPPER_SNAKE_CASE
- **Modules/Packages:** snake_case
- **PyTorch:** PascalCase with suffix 'Model' (Eg. `ClassifierModel()`)

4 Code Layout & Structure

4.1 Indentation

- **Frontend:** 2 spaces
- **Backend:** 4 spaces

4.2 Line Length

- Between 100-120 chars per line

4.3 Quotation Marks

- **Frontend:** Single quotes (')
- **Backend:** Double quotes (")

4.4 Braces

- Same-line opening brace (Eg. function func() {)

4.5 Comments

- Add comments to code that may be difficult for others to understand.

4.6 Imports (Python)

- Standard libraries → third-party → local; each group separated by a blank line.

5 Frontend Standards

5.1 TypeScript

- Style guide baseline: **Airbnb TypeScript Style Guide**; enforced via ESLint.
- The use of functional components and hooks are preferred over class components.
- Type everything explicitly at module boundaries (props, return types, API responses); avoid using `any`.
- One component per file; the file name matches the component name.

5.2 DaisyUI (Styling)

- Use DaisyUI component classes on top of Tailwind CSS rather than writing custom CSS from scratch; only add custom styles if DaisyUI has no equivalent.
- Stick to DaisyUI's theme tokens (spacing, colouring) rather than hardcoding raw values to keep the app consistent if the theme changes.
- Keep utility classes readable: for components with many classes, extract them into a variable or using `class-variance-authority`/similar rather than one long inline string.

5.3 Playwright (Testing)

- End-to-end and functional tests are located in the `app/frontend/testing` folder.
- **File naming convention:** `*.spec.ts`
- Test files should mirror the page/flow being tested (Eg. `login.spec.ts`).
- Use role/text-based selectors (`getByRole`, `getByText`).
- Tests should be independent and should not rely on execution order.

6 Backend Standards

6.1 Python

- Style guide baseline: **PEP8**, enforced via Black (formatting) and Pylint (linting).
- Use type hints on function signatures.
- Use NumPy-style docstrings for public data science focused functions/classes and Google-style docstrings for non-data science focused functions/classes for any machine learning code.

```python
def calculate_velocity(distance: float, time: float) -> float:
    """Calculates the average velocity of an object.

    Args:
        distance (float): The total distance traveled in meters.
        time (float): The total time taken in seconds. Must be greater than
            zero.
    Returns:
        float: The calculated velocity in meters per second.
    Raises:
        ValueError: If the time provided is less than or equal to zero.
    """
    if time <= 0:
        raise ValueError("Time must be greater than zero.")
    return distance / time
```

Example: Google-style docstring

```python
def calculate_velocity(distance: float, time: float) -> float:
    """Calculates the average velocity of an object.

    Parameters
    ----------
    distance : float
        The total distance traveled in meters.
    time : float
        The total time taken in seconds. Must be greater than zero.

    Returns
    -------
    float
        The calculated velocity in meters per second.

    Raises
    ------
    ValueError
        If the time provided is less than or equal to zero.
    """
    if time <= 0:
        raise ValueError("Time must be greater than zero.")
    return distance / time
```

Example: NumPy-style docstring

6.2 Pytest (Testing)

- Test files are found at `app/backend/testing`.
- **File naming convention:** `test_*.py`
- One assertion concept per test
- **Test naming:** `test_<function>_<scenario>_<expected_result>` (Eg. `test_get_user_data_invalid_id_raises_e`

7 AI/ML Standards

- Style guide baseline: same as backend.
- Set and document random seeds for reproducibility in training scripts.
- Log hyperparameters and metrics consistently.
- Keep data preprocessing, training, and inference code in separate modules (Eg. `data/`, `training/`, `inference/`).
- Include tests that validate model input/output shapes and basic sanity checks (Eg. Output is within an expected range)

8 API Design Guidelines

- Follow **RESTful conventions:** resources as nouns, HTTP verbs for actions (`GET /users/id`, `POST/users`).
- Request/response bodies use `camelCase` JSON keys at the API boundary (to match frontend conventions), even though the backend internally uses `snake_case`.
- Use standard HTTP status codes correctly (`400` for validation errors, `403/403` for auth, `404` for missing resources, `500` for server errors).
- Document all endpoints (using Swagger) and keep the spec in sync with implementation.
- Paginate list endpoints; never return unbounded result sets.

9 Error Handling & Logging

- Never silently swallow exceptions/errors - log or handle explicitly. Include enough context in error messages/logs to debug without reproducing locally.
- Fail fast where appropriate; avoid deeply nested error-handling logic.
- **Frontend:** Surface user-facing errors via a consistent error boundry/toast pattern; log unexpected errors to the console/monitoring service.
- **Backend/ML:** Raise typed/custom exceptions rather than generic `Exception`; catch and translate them to appropriate API error responses at the boundary.
- Never log secrets, credentials or sensitive user data.

10 Configuration & Secrets

- All configuration is environment-based and never hardcoded (`.env` files locall, environment variables in deployed environments).
- Secrets are never committed to the repository. Use `.env.example` to document required variables without values.
- Config values are validated at service startup; fail fast if a required variable is missing.
- Scan commits with secret scanners

11 Configuration Management

Tool	Purpose	Applies To	Config File
ESLint	Lint: TypeScript/React	Frontend	`frontend/.eslintrc.*`
Prettier	Formatter: TypeScript/React	Frontend	`frontend/.prettierrc`
DaisyUI	Component Styling	Frontend	`frontend/tailwind.config.js`
Playwright	Testing	Frontend	`frontend/playwright.config.ts`
Black	Formatter: Python	Backend, AI/ML	`backend/`
Pylint	Lint: Python	Backend, AI/ML	`backend/.pylintrc`
Pytest	Testing	Backend, AI/ML	`backend/testing`
GitHub Actions	Automation	All	`.github/workflows/*.yml`

12 Code Reviews & Pull Requests

12.1 Review Requirements

- Every change is to be submitted via a pull request; No direct commits to `main`.
- All pull requests require at least one approval from a team member.
- Ensure that an adequate description is provided of what your code does in your pull request; add additional information that is relevant as well as run commands where possible.
- Pull requests must pass the code quality gate
- Each pull request should be associated with only one issue as far as possible.
- The author of the pull request is required to respond to review comments before requesting a re-review.
- All CI/CD checks must pass before a pull request can be merged into `main`.

12.2 Review Guidelines

- Reviewers are required to check for correctness, clarity and adherence to this document's standards.
- Ensure code documentation is adequate.
- Improvements can be suggested but ensure they are constructive.

13 CI/CD Pipeline Standards

13.1 GitHub Actions

- The CI/CD Pipeline is defined in `.github/workflows/`
- Run all linting, formatting checks and execute full test suite before merging.
- Coverage reports should be generated and uploaded. Builds must fail if any checks fail.

13.2 Deployment

- Docker containers are used to keep environments consistent.
- Use a blue-green deployment strategy.
- Ensure rollbacks are possible
- Use environment-specific configurations
- Monitor deployment health and performance.

14 File Structure & Organisation

14.1 Repository Structure

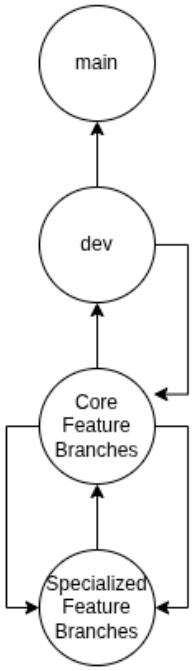
```
AI-Powered-Fire-Spread-Prediction-and-Containment-System/
├── .github/
│   └── workflows/
├── app/
│   ├── backend/
│   │   ├── src/
│   │   │   ├── ai/
│   │   │   ├── dependencies/
│   │   │   ├── enums/
│   │   │   ├── models/
│   │   │   ├── services/
│   │   │   ├── schemas/
│   │   │   └── routes/
│   │   ├── tests/
│   │   ├── db.py
│   │   ├── main.py
│   │   ├── package.json
│   │   └── requirements.txt
│   ├── frontend/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── pages/
│   │   │   ├── types/
│   │   │   ├── styles/
│   │   │   └── middleware.ts
│   │   ├── public/
│   │   │   └── images/
│   │   ├── testing/
│   │   ├── next.config.js
│   │   ├── playwright.config.ts
│   │   ├── tailwind.config.js
│   │   ├── package.json
│   │   └── tsconfig.json
│   └── ml/
│       ├── training/
│       └── datasets/
├── database/
│   ├── minio/
│   └── postgres/
├── docs/
├── docker-compose.test.yml
├── docker-compose.yml
└── README.md
```

14.2 File Naming Conventions

- **React Component:** PascalCase
- **Pages:** camelCase
- **Python Files:** snake_case

15 Git Repository & Strategy

We use the Git Flow branching strategy to allow for development in parallel while keeping our production stable.



15.1 Maintained Branches

- main (production)
- dev (integration)
- Core feature branches (admin, firefighters, users, guests, ai_model)

15.2 Branch Purposes

- **main :** Production ready. Dev is merged into main when it is stable and fully tested.
- **dev :** This is where code from the core feature branches are integrated to create a stable version of our system before being pushed to main.

- **Core feature branches :** These all branch off of dev and allow for us to work on a specific category.
- **Specialized feature branches :** These branches branch from the core feature branches. This allows us to work on small, specific features of the the core feature branches. (Eg. A specialized branch, sidebar, branching from the admin core feature branch implies that a sidebar is being developed for admin)

15.3 Branch Names

- Specialized branches must use the following naming convention: `<featureName>_<yourName>` (Eg. `sidebarNav_alice`)

15.4 Merging

- Ensure to delete specialized branches once they have been successfully merged into a core feature branch.

16 Conclusion

All collaborators are expected to follow the standards laid out in this document to keep the codebase consistent, easier to review, and easier to maintain as a team as the project grows.

For questions/queries about this document, please contact the team leads at:
panicatthekernel.26@gmail.com

17 Change Log

Date	Version	Change
July 2026	1.0	Initial version
August 2026	2.0	File Structure Update